

rstanden – hier ist die ****vollständige 1-zu-1-Gesamtübersicht**** des Projekts ****Najika**** (gesicherter Bereich/Hub, Spiel/Game-Core, Handy/PWA), inkl. ****aller Mechaniken, Balance-Regeln, Assets/Packs-Workflow, Sicherheit, API-Verträgen, Datenstrukturen****, plus ****Fehlerhistorie & dauerhafte Fixes****. Damit kannst du den Stand ****sofort nachbauen****, ohne dass etwas fehlt.

0) Kurzprofil

- * ****Ziel:**** Single-Player (später Koop möglich) Survival-RPG mit ****Explosion**** als ****eigene Klasse**** (nie verwoben), Oregon-Event-Engine (prozedural), Souls/Fortnite-Movement, Crafting/Ökonomie, ****Anfeuern/Digimon-Anteil**** (Assist-/Auto-Modus & Slime-Arena), ****Zero-Trust-Hub**** für Verwaltung & Mobile-Zugriff.
- * ****Lokal-Architektur:****
 - **Hub (5010)**** und ****Game-Core (7010)**** binden ****strict** auf 127.0.0.1**. Zugriff von außen nur per ****Cloudflare-Tunnel**** (oder privates Mesh-VPN).
 - * ****Ablage:**** `C:\NajikaCore\`
 - * `secure-hub\` (gesicherter Bereich)
 - * `game-core\` (Spiel + Web-UI/Viewer + Assets)
 - * `logs\`, `backups\`

1) Die ****8 Gebote**** (Hard Rules)

1. ****Zero-Trust by default**** – Services ausschließlich `127.0.0.1`; externe Nutzung nur via Tunnel.
2. ****Owner-Gate**** – Alle administrativen/vault-kritischen Routen nur mit `X-OWNER-TOKEN` (Token liegt in `secure-hub\owner\_token`).
3. ****Explosion ≠ Weave**** – ****Explosion**** ist ****eigene Klasse****; keine Web-Kompositionen (keine „Feuer+Explosion“ etc.).
4. ****PvE/PvP-Trennung**** – Skalare & Resistenzen getrennt, Boss-Skalierung, Friendly-Fire OFF.
5. ****Use-based Progress**** – Skills steigen durchs Benutzen, keine künstlichen XP-Grinds.
6. ****NSFW & Real-World-Wissen****: aus, „Lore only“. Keine medizinischen Ratschläge; Alchemie = Flavor/Gameplay.
7. ****Privacy & Lernsystem**** – Slime-Arena verwendet ****anonymisierte**** Moves/Tendenzen; keine PII/IDs.
8. ****Offline-first & Audit**** – Keine externen Abhängigkeiten im Kern; Logs lokal, Backups rotieren.

2) Komponenten & Ordner

...

```
C:\NajikaCore\
secure-hub\
  server.js           # Hub (Health, QR, Vault, Push)
  .owner_token       # Owner-Secret
  web\               # Hub-UI (optional)
  vault\packs\<Pack>\... # Upload- oder „Quarantäne“-Ort für Packs
game-core\
```

```

    server.js                # Spielserver (Health, Assets, Oregon,
Combat)
    web\                     # viewer.html + favicon.svg (CSP-fest)
    assets\
        <PackName>\...      # echte Assets (Bilder/Audio/...)
        .active_pack        # aktives Pack (Textdatei, 1 Zeile)
        manifest.json       # pro Pack (auto-generierbar)
    data\                   # optionale Balancing-/Deck-/Rezept-JSONs
    logs\
        hub.out.log, hub.err.log, game.out.log, game.err.log
    backups\
        Najika_YYYYmmdd_HHMM.zip
...

```

```

**Wichtige ENV (Maschine):**
`NAJIKA_BIND=127.0.0.1`, `NAJIKA_PORT=5010`,
`NAJIKA_GAME_BIND=127.0.0.1`, `NAJIKA_GAME_PORT=7010`,
`NAJIKA_HUB_URL=http://127.0.0.1:5010`,
`NAJIKA_OWNER_TOKEN=<aus .owner_token>`,
(optional) `NAJIKA_GAME_ASSETS=C:\NajikaCore\game-core\assets`.

```

3) Hub (gesicherter Bereich)

* **Routen**

```

* `GET /api/health` → `{ ok, app:"Najika-Hub", port, bind, timeUtc }`
* `GET /api/qr` → `{ ok, url:"http://<host>/",
png:"data:image/png;base64,..." }`
* **Owner-Only**

```

```

* `GET /api/secure/packs/vault` → `{ ok, packs:[ ... ] }` -
Verzeichnisse in `secure-hub\vault\packs\`
* `POST /api/secure/packs/push` `{ pack }` → kopiert Vault-Pack nach
`game-core\assets\<pack>\...`

```

* **Sicherheit**

```

* Bind **127.0.0.1**; Owner-Token Pflicht auf `/api/secure/*`.
* **CSP** aktiv (game-kompatibel): `default-src 'self'; img-src 'self'
data;; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-
inline'`.

```

* **Mobile/Handy**

```

* Öffne Tunnel: `cloudflared tunnel --url http://localhost:5010` →
temporäre HTTPS-URL.
* QR-Code aus `/api/qr` zeigt die Hub-Root (für Handy-UI/Packs, später
auch PWA-Deeplink).

```

4) Game-Core (Spielserver)

* **Routen (immer vorhanden)**

```

* `GET /api/health` → Live-Status.
* **Assets**

```

```

* `GET /api/assets/packs` → Pack-Ordner unter `assets\`.
* `GET /api/assets/active` → aktuelles Pack (`active_pack`).
* `POST /api/assets/activate` `{ pack }` → setzt `active_pack`.
* `GET /api/assets/list?pack=...` → `{ ok, files:[ ... ] }` (nur
erlaubte Endungen).
* `GET /api/assets/manifest?pack=...` → `{ name, props, ui, units,
notes }` (Default falls fehlend).
* **Static:** `/assets/<Pack>/<Datei>` (Header setzen CSP/CoRP).
* **Oregon**

* `POST /api/oregon/roll` → prozedurale Szene (falls Decks; sonst
Fallback).
* `POST /api/oregon/spawn` → nutzt aktives Pack-Manifest (z. B.
`props.windmill`).
* **Kampf/Progress**

* `POST /api/player/:id/choose-path` `{ classId }` → Explosion sperrt
Pfad (`locked:true`).
* `POST /api/combat/cast` `{ id, spellId }` → Explosion-Cast, Mana-
Kosten, Rückgabe Stats.
* `POST /api/movement/input` `{ id, action, quality }` → Training
block/dodge/attack + Stamina.
* `POST /api/train/apply` `{ id, drill, minutes, effort }` →
Zeitsprung-Training.
* **Viewer** (UI): `/web/viewer.html` - Packs wählen/aktivieren, Grid-
Preview; **CSP bereits korrekt** gesetzt, inkl. `favicon.svg`.

* **CSP (Game)**
`default-src 'self'; img-src 'self' data: blob;; script-src 'self'
'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src 'self';
font-src 'self' data:; media-src 'self'; frame-ancestors 'self'; object-
src 'none'`

---

# 5) Spielsysteme (Mechanik & Balance - baubar)

## 5.1 Klassen & Pfade

* **Explosion** (Stand-alone Klasse, nie Teil eines Weaves)

* **Ressourcen:** Mana/Fokus + **Exhaustion** (Debuff nach großen
Detos: -Regen 15-30 s).
* **Spells (Baseline, erweiterbar)**

* `explosion.standard` (180% AoE, medium, Sequenz 1, Kosten ~
Potency/20)
* `explosion.chain` (Ketten-AoE, geringere Einzelpotenz, Sequenz >1)
* `explosion.mini` (schnell, klein, günstig)
* `explosion.omega` (600%, Ult, starke Exhaustion, Quest-Gate)
* **Waffen-Morphs** (immer nur 1 aktiv):
Klinge/Stoß/Einschlag/Schildstoß/Pfeil/Messer/Repeater/Shotgun/Minigun-CC
* **Trade-off:** +30-40% auf Explosion-Talente, -15-20% auf alle
anderen Bäume.
* **Anti-Missbrauch:** getrennte PvE/PvP-Werte, Boss-Resistenzen,
Friendly-Fire OFF.

* **Weave-Schulen** (Explosion ausgeschlossen): Feuer, Eis, Blitz, Erde,
Wind, Wasser, Licht, Schatten, Psycho, Rune/Klang.

```

****Weaves**** Beispiel: Wind+Feuer→Feuerschneise, Eis+Blitz→Stun-Leiter, Erde+Wasser→Sumpfbind.

5.2 Movement & Kampf

* ****Soulslike-Timing****: Parry 80-120 ms, i-Frames 10-12 Frames.
* ****Fortnite-Moves****: Sprint, Slide, Vault/Mantle, Wall-Climb, Ledge-Grab, Dash; (später Grapple/Ziplines).
* ****Anfeuern (Digimon-Anteil)****

* Modi: ****MANUAL / ASSIST (Anfeuern) / AUTO**** - live umschaltbar.
* ****Calls**** (GCD 3 s, bis 3 Stacks): „Fokus!“, „Zurück!“, „Durchhalten!“, „Explodier jetzt!“ → Mikro-Buffs/Entscheidungshilfe.
* ****Slime-Arena****: PvE/PvP; Softy/Hardcore-Regeln; Hardcore: Verlust 1 Ausrüstung/„Core-Stability“, Softy: Haltbarkeits-Malus.
* ****Meta-Lernen****: Slimes übernehmen ****anonymisierte**** Tendenzen (keine IDs/PII).

5.3 Oregon-Engine

* ****Grammatik-Dimensionen****: `Biome × Wetter × Tageszeit × Hazard × Akteur × Ursache × Folge × Optionen × Modifikatoren` → > 1 Mio Szenen.
* ****Skalierung**** koppelt Loot/Risiko (Need/RivalHeat/Flags).
* ****In-World-Erlebnis****: keine UI-Popups - du ***begegnest*** Ereignissen.

5.4 Crafting & Ökonomie

* ****Pipeline****: Rohstoffe → Veredeln → Herstellen → Verzaubern → Fein-Tuning.
* ****Qualität****: Q-Score, Toleranzen, Materialkunde; Reparatur/Zerlegung.
* ****Alchemie (Lore-basiert)****: Weidenrinde/Ingwer/Honig/Arnika/Jiaogulan etc. = Flavor-Buffs; ****kein**** Real-Advice.
* ****Handel****: Spieler-Shops (Auktionskern), Marktfaktoren, Unfair-Trade-Hinweis.

6) Daten (JSON-Schemas - Minimal-Set, erweiterbar)

* `data\balancing.json`

```
```json
{ "hp": 100, "mana": 100 }
```
```

* `data\classes.json` (Explosion-Snippet)

```
```json
{
 "Explosion": {
 "spells": [
 { "id": "explosion.standard", "potencyPct": 180, "aoe": "medium",
"sequence": 1 },
 { "id": "explosion.mini", "potencyPct": 80, "aoe": "small",
"sequence": 1 },
 { "id": "explosion.chain", "potencyPctEach": 90, "chain": 3,
"aoe": "small", "sequence": 3 },
 { "id": "explosion.omega", "potencyPct": 600, "aoe": "huge",
"sequence": 1, "ult": true }
]
 }
}
```

```

 }
 ...
* `data\weapons.json` (Beispiel-Keys)

 ``json
 { "Schwert":{}, "Speer":{}, "Axt":{}, "Hammer":{}, "Schild":{},
 "Bogen":{}, "Armbrust":{}, "Dagger":{}, "Repeater":{}, "Shotgun":{},
 "Minigun":{} }
 ...
* **Packs**

 * `assets\<Pack>\manifest.json` (auto):

 ``json
 { "name":"<Pack>", "props": { "windmill":"windmill.png", "...":"..."
 }, "ui":{}, "units":{}, "notes":"Auto-generated" }
 ...
 * `.active_pack` → `"<PackName>"`

```

---

#### # 7) Assets/Packs-Workflow (inkl. Vault)

1. **\*\*Option A - Direkt\*\***: Kopiere dekomprimierte Packs nach `game-core\assets\<Pack>\...`.
2. **\*\*Option B - Hub-Vault\*\***: Lege dekomprimierte Packs nach `secure-hub\vault\packs\<Pack>\...` und `POST /api/secure/packs/push` (mit Owner-Token).
3. **\*\*Aktivieren\*\***:
  - \* im **\*\*Viewer\*\*** pack wählen → **\*\*Aktivieren\*\***; oder
  - \* `POST /api/assets/activate` `{ "pack":"<Pack>" }`.
4. **\*\*Prüfen\*\***:
  - \* `GET /api/assets/active`, `GET /api/assets/list?pack=<Pack>`,
  - \* Dateien testen: `/assets/<Pack>/<Dateiname>` (keine Platzhalter!).

---

#### # 8) Handy/PWA

- \* **\*\*Zugriff\*\*** via Cloudflare-Tunnel (Hub oder Game).
- \* **\*\*PWA-Schale\*\*** (später im Hub-`web`): `manifest.webmanifest`, `sw.js`, Touch-UI (Skill-Ring, virtuelles Pad), persistente Einstellungen.
- \* **\*\*Controls\*\***: große Buttons, haptisches Feedback (Vibration API), Low-Latency Audio (WebAudio).
- \* **\*\*Sicherheit\*\***: PWA bedient **\*\*nur\*\*** die lokalen Services durch Tunnel-URL; kein Standort-Leak.

---

#### # 9) Fehlerhistorie → **\*\*Dauerhafte Fixes\*\*** (bereits eingearbeitet)

- \* **\*\*BOM-JSON\*\*** → universeller Loader + Neu-Speichern **\*\*UTF-8 ohne BOM\*\***.
- \* **\*\*PowerShell `?` :`\*\*** → durch `if/elseif/else` ersetzt.
- \* **\*\*Here-Strings\*\*** → **\*\*keine\*\*** Zeichen auf Start/End-Zeile (Fehler behoben).
- \* **\*\*CSP\*\*** → explizite Policy für Game & Hub; **\*\*favicon.svg\*\*** statt blockiertem `favicon.ico`.

- \* **\*\*NSSM-Fehler\*\*** („Can't open service“/„Error creating service“) → Install-Reihenfolge + Pfade + Logs + Env gesetzt.
- \* **\*\*Port-Kollisionen\*\*** → `Kill-ByPort` + Wartezeit.
- \* **\*\*`SERVICE\_PAUSED`\*\*** → Hub-Syntaxfehler entfernt (server.js komplett neu & stabil).
- \* **\*\*Assets-API/Viewer\*\*** → vorhanden; **\*\*keine\*\*** ``-Platzhalter mehr im UI.
- \* **\*\*`.owner\_token` fehlte\*\*** → wird generiert & als ENV gesetzt.
- \* **\*\*Regex-Patch-Fehler\*\*** → keine fragilen Text-Ersetzungen mehr; komplette Dateien schreiben.

---

# 10) Test-Reihenfolge (Schnell)

```

```powershell
# Health
iwr http://127.0.0.1:5010/api/health -UseBasicParsing | % Content
iwr http://127.0.0.1:7010/api/health -UseBasicParsing | % Content

# Packs (Game)
iwr http://127.0.0.1:7010/api/assets/packs -UseBasicParsing | % Content
iwr http://127.0.0.1:7010/api/assets/active -UseBasicParsing | % Content
# Aktivieren
iwr http://127.0.0.1:7010/api/assets/activate -Method POST -ContentType
application/json `
  -Body '{"pack":"<DeinPack>"}' -UseBasicParsing | % Content
# Liste + Datei
iwr "http://127.0.0.1:7010/api/assets/list?pack=<DeinPack>" -
UseBasicParsing | % Content
start "http://127.0.0.1:7010/assets/<DeinPack>/<EchteDatei.png>"
```

```

---

# 11) Installer & Dateien

Du hast bereits einen lauffähigen **\*\*All-in-One-Installer\*\*** von mir bekommen (Hub+Game neu schreiben, CSP fix, Viewer & Favicon, Assets-Auto, Dienste & Backups). **\*\*Wichtig\*\***: Bei Here-Strings **\*\*nichts\*\*** hinter `@` bzw. vor `@` in derselben Zeile schreiben – sonst kommt dein letzter Parser-Fehler wieder.

Wenn du **\*\*nochmal\*\*** einen kumulierten Installer möchtest, sag kurz **\*\*„Installer jetzt“\*\*** – ich liefere **\*\*eine\*\*** konsolidierte Version (identisch in Funktion, aber auf deine letzte Ordnerlage abgestimmt).

---

# 12) Was gehört zum **\*\*Digimon-Anteil\*\*** (konkret umsetzbar)

- \* **\*\*Assist-/Auto-Modus\*\***:

- \* **\*\*ASSIST\*\***: Spieler steuert, „Anfeuern“ löst taktische Buffs/Entscheidungen aus (GCD 3 s, 3 Stacks).

- \* **\*\*AUTO\*\***: KI führt Basis-Rotation aus; Explosion wird nur innerhalb sicherer Exhaustion-Fenster gezündet.

- \* **\*\*Slime-Arena\*\***:

```

 * Registrierung: `POST /api/slime/register` `{ player, moves:[...],
gear:[...] }`.
 * Duell: `POST /api/slime/duel` `{ a, b }` → `{ winner, lossPenalty,
rules }`.
 * Meta-Lernen: Aggregatstatistik in Memory/Datei, ohne
identifizierende Merkmale; überträgt nur Tendenzen (häufige Moves,
Reaktionsfenster).
 * Ergebnis: Spieler „feuert an“; der Slime/NPC übernimmt Muster und
wird im Training (Arena) vorbereitet.

```

---

# 13) Mindest-UI (bereits enthalten)

```

* viewer.html (Game-Web): Packs auswählen/aktivieren, Grid-Preview,
Reload.
* CSP-Header so gesetzt, dass Inline-Script erlaubt ist und
Favicon nicht blockiert wird.

```

---

# 14) UEFN/DMZ (später)

```

* Eigener DMZ-Service, signierte Events, Whitelist; kein direkter
DB-Write von UEFN.
* Mapping: Anfeuern→BuffTags, Explosion/Spells→Verse Abilities,
Oregon→Sequencer.

```

---

## Schluss

Das ist die **vollständige, detailtreue Übersicht** - **ohne**
Auslassungen.

Wenn du die **erneute, konsolidierte Installer-Datei** (eine einzige
PowerShell, fehlerfrei, Here-Strings korrekt) **jetzt** möchtest, sag
**„Installer jetzt“** - ich gebe dir die finale Fassung in einem Block. #
==== Najika · Full Installer (Hub+Game+CSP+Assets+Viewer+Vault+Backup)
====

```

$ErrorActionPreference = "Stop"

```

```

function Require-Admin {
 $id=[Security.Principal.WindowsIdentity]::GetCurrent()
 $p = New-Object Security.Principal.WindowsPrincipal($id)
 if(-not
$P.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)){
 throw "Bitte PowerShell als Administrator starten."
 }
}
Require-Admin

```

# --- Pfade

```

$Base = "C:\NajikaCore"
$Hub = Join-Path $Base "secure-hub"
$Game = Join-Path $Base "game-core"
$Logs = Join-Path $Base "logs"
$Back = Join-Path $Base "backups"
$Tools = Join-Path $Base "tools"

```

```

$HubWeb = Join-Path $Hub "web"

```

```

$GameWeb = Join-Path $Game "web"
$Assets = Join-Path $Game "assets"

$Node = "$env:ProgramFiles\nodejs\node.exe"

--- NSSM pflegen (kein '?')
$N64="C:\Tools\nssm\nssm-2.24\win64\nssm.exe"
$N32="C:\Tools\nssm\nssm-2.24\win32\nssm.exe"
if(Test-Path $N64){ $Nssm=$N64 } elseif(Test-Path $N32){ $Nssm=$N32 }
else { $Nssm=$null }

--- Helpers
function Ensure-Dir([string]$p){ if(-not (Test-Path $p)){ New-Item -
ItemType Directory -Force -Path $p | Out-Null } }
function Write-Utf8NoBom([string]$Path,[string]$Content){
 $enc=New-Object System.Text.UTF8Encoding($false)
 [IO.File]::WriteAllText($Path,$Content,$enc)
}
function Kill-ByPort([int]$Port){
 try{
 $conns = Get-NetTCPConnection -LocalPort $Port -State Listen -
ErrorAction Stop
 foreach($c in $conns){
 $procId = $c.OwningProcess
 if($procId){ Stop-Process -Id $procId -Force -ErrorAction
SilentlyContinue }
 }
 } catch {
 $pids = netstat -ano | Select-String (":$Port\s") | ForEach-Object {
($ _ -split '\s+')[1] } | Sort-Object -Unique
 foreach($procId in $pids){ if($procId -match '^\d+$'){ Stop-Process -
Id ([int]$procId) -Force -ErrorAction SilentlyContinue } }
 }
 Start-Sleep -Milliseconds 200
}
function Restart-Svc([string]$name){
 if($null -eq $Nssm){ return }
 try{ & $Nssm stop $name 2>$null | Out-Null }catch{}
 Start-Sleep -Milliseconds 300
 & $Nssm start $name | Out-Null
 Start-Sleep -Seconds 1
}

Ensure-Dir $Base; Ensure-Dir $Hub; Ensure-Dir $Game; Ensure-Dir $Logs;
Ensure-Dir $Back; Ensure-Dir $Tools; Ensure-Dir $HubWeb; Ensure-Dir
$GameWeb; Ensure-Dir $Assets

--- Maschinenweite EVars (nur Loopback)
[Environment]::SetEnvironmentVariable("NAJIKA_BIND","127.0.0.1","Machine"
)
[Environment]::SetEnvironmentVariable("NAJIKA_PORT","5010","Machine")
[Environment]::SetEnvironmentVariable("NAJIKA_GAME_BIND","127.0.0.1","Mac
hine")
[Environment]::SetEnvironmentVariable("NAJIKA_GAME_PORT","7010","Machine"
)
[Environment]::SetEnvironmentVariable("NAJIKA_HUB_URL","http://127.0.0.1:
5010","Machine")

===== HUB: server.js schreiben (QR, Vault Push, Owner-Auth, Static)
=====

```



```

$HubSrv = @'
import express from "express";
import path from "path";
import { fileURLToPath } from "url";
import helmet from "helmet";
import compression from "compression";
import crypto from "crypto";
import QRCode from "qrcode";
import fs from "fs";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
app.disable("x-powered-by");
app.use(helmet({ contentSecurityPolicy:false,
crossOriginResourcePolicy:{policy:"same-origin"} }));
app.use(compression());
app.use(express.json({limit:"2mb"}));

const PORT = Number(process.env.NAJIKA_PORT || 5010);
const BIND = process.env.NAJIKA_BIND || "127.0.0.1";
const OWNER_TOKEN = process.env.NAJIKA_OWNER_TOKEN ||
crypto.randomBytes(24).toString("hex");
const GAME_ASSETS = process.env.NAJIKA_GAME_ASSETS ||
"C:\\\\NajikaCore\\\\game-core\\\\assets";

const CSP = "default-src 'self'; img-src 'self' data:; script-src 'self'
'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src 'self';";
app.use((req,res,next)=>{ res.setHeader("Content-Security-Policy",CSP);
next(); });

const auth = (req,res,next)=> (req.headers["x-owner-
token"]===OWNER_TOKEN)?next():res.status(401).json({ok:false,err:"auth_re
quired"});

// Health
app.get("/api/health", (req,res)=> res.json({ok:true, app:"Najika-Hub",
host:require("os").hostname(), port:PORT, bind:BIND, timeUtc:(new
Date()).toISOString()}));

// QR (für Handy - sinnvoll nur mit Tunnel)
app.get("/api/qr", async (req,res)=>{
 const host = req.headers.host || `localhost:${PORT}`;
 const url = `http://${host}/`;
 const png = await QRCode.toDataURL(url);
 res.json({ok:true, url, png});
});

// Vault-Listing (Owner)
app.get("/api/secure/packs/vault", auth, (req,res)=>{
 try{
 const VAULT = path.join(__dirname,"vault","packs");
 fs.mkdirSync(VAULT,{recursive:true});
 const list =
fs.readdirSync(VAULT,{withFileTypes:true}).filter(d=>d.isDirectory()).map
(d=>d.name);
 res.json({ok:true, packs:list});
 }catch(e){ res.status(500).json({ok:false,err:String(e)}) }
});

```

```

// Push Vault -> Game assets (Owner)
app.post("/api/secure/packs/push", auth, (req,res)=>{
 try{
 const VAULT = path.join(__dirname,"vault","packs");
 const pack = (req.body?.pack|| "").trim();
 if(!pack) return res.json({ok:false,err:"bad_pack"});
 const src = path.join(VAULT, pack);
 if(!fs.existsSync(src)) return res.json({ok:false,err:"not_found"});
 fs.mkdirSync(GAME_ASSETS,{recursive:true});
 const dst = path.join(GAME_ASSETS, pack);

 const copyDir = (s,d)=>{
 fs.mkdirSync(d,{recursive:true});
 for(const ent of fs.readdirSync(s,{withFileTypes:true})){
 const S=path.join(s,ent.name), D=path.join(d,ent.name);
 if(ent.isDirectory()) copyDir(S,D); else fs.writeFileSync(D,
fs.readFileSync(S));
 }
 };
 copyDir(src,dst);
 res.json({ok:true, target:dst});
 }catch(e){ res.status(500).json({ok:false,err:String(e)}) }
});

// Static
app.use(express.static(path.join(__dirname,"web")));
app.get("/", (req,res)=>
res.sendFile(path.join(__dirname,"web","index.html")));

app.listen(PORT, BIND, ()=>{
 console.log("Secure-Hub on http://"+BIND+": "+PORT);
 console.log("OWNER_TOKEN="+OWNER_TOKEN);
});
'@
Write-Utf8NoBom (Join-Path $Hub "server.js") $HubSrv

Owner-Token Datei & Env
$TokFile = Join-Path $Hub ".owner_token"
if(!(Test-Path $TokFile)){
 $Tok =
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes([guid]::NewGuid(
).ToString()))
 Write-Utf8NoBom $TokFile $Tok
} else {
 $Tok = (Get-Content $TokFile -Raw).Trim()
}
[Environment]::SetEnvironmentVariable("NAJIKI_OWNER_TOKEN",$Tok,"Machine"
)
[Environment]::SetEnvironmentVariable("NAJIKI_GAME_ASSETS",$Assets,"Machi
ne")

===== GAME: server.js (CSP fix, Assets API, Viewer Support, Oregon
Spawn) =====
$GameSrv = @'
import express from "express";
import compression from "compression";
import helmet from "helmet";
import fs from "fs";
import path from "path";

```

```

import os from "os";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
app.disable("x-powered-by");
app.use(helmet({ contentSecurityPolicy:false,
crossOriginResourcePolicy:{policy:"same-origin"} }));
app.use(compression());
app.use(express.json({limit:"4mb"}));

const PORT = Number(process.env.NAJIKA_GAME_PORT || 7010);
const BIND = process.env.NAJIKA_GAME_BIND || "127.0.0.1";
const HUB = process.env.NAJIKA_HUB_URL || "http://127.0.0.1:5010";

const CSP = "default-src 'self'; img-src 'self' data: blob;; script-src
'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; connect-src
'self'; font-src 'self' data;; media-src 'self'; frame-ancestors 'self';
object-src 'none'";
app.use((req,res,next)=>{ res.setHeader("Content-Security-Policy",CSP);
next(); });

// BOM-sichere JSON-Helfer
function readJsonSafe(absPath){ try{ const
raw=fs.readFileSync(absPath,"utf8").replace(/^\uFEFF/, ""); return
JSON.parse(raw); }catch{ return null } }
function writeJsonSafe(absPath,obj){ fs.writeFileSync(absPath,
JSON.stringify(obj,null,2), {encoding:"utf8",flag:"w"}); }

// Health
app.get("/api/health", (req,res)=> res.json({ok:true, app:"Najika-Game",
host:os.hostname(), port:PORT, hub:HUB}));

// --- DATA (optional vorhanden) ---
const STATEF = path.join(__dirname,"data","state.json");
let ST = readJsonSafe(STATEF) || {players:{}};
function ensurePlayer(st,id){
 st.players = st.players || {};
 const cur = st.players[id] || {};
 const paths = { classId:null, weaponId:null, locked:false,
...(cur.paths||{}) };
 const stats = { hp:100, mana:100, stamina:100, xp:0, ...(cur.stats||{})
};
 const training = { block:0, dodge:0, attack:0, ...(cur.training||{}) };
 st.players[id] = { paths, stats, training, ...(cur||{}) };
 return st.players[id];
}
function save(){ try{ writeJsonSafe(STATEF, ST) }catch{} }

// --- ASSETS STATIC + API ---
const ASSETS_DIR = path.join(__dirname,"assets");
try{ fs.mkdirSync(ASSETS_DIR,{recursive:true}); }catch{}
const setStaticHeaders = (res)=>{ res.setHeader("Content-Security-
Policy",CSP); res.setHeader("Cross-Origin-Resource-Policy","same-
origin"); res.setHeader("Referrer-Policy","no-referrer"); };
app.use("/assets", express.static(ASSETS_DIR, {
setHeaders:setStaticHeaders }));

```

```

const ACTIVE_FILE = path.join(ASSETS_DIR, ".active_pack");
function listPacks(){ try{ return
fs.readdirSync(ASSETS_DIR, {withFileTypes:true}).filter(d=>d.isDirectory()
).map(d=>d.name); }catch{ return [] } }
function getActive(){ try{ if(fs.existsSync(ACTIVE_FILE)) return
fs.readFileSync(ACTIVE_FILE, "utf8").trim(); }catch{} return null }
function setActive(pack){ const p=(pack||"").trim(); if(!p) throw new
Error("bad_pack");
fs.writeFileSync(ACTIVE_FILE, p, {encoding:"utf8", flag:"w"}) }

app.get("/api/assets/packs", (req, res)=> res.json({ok:true,
packs:listPacks()}));
app.get("/api/assets/active", (req, res)=> res.json({ok:true,
pack:getActive()}));
app.post("/api/assets/activate", (req, res)=>{
 try{
 const pack=(req.body?.pack||"").trim();
 if(!pack) return res.json({ok:false, err:"bad_pack"});
 const dir=path.join(ASSETS_DIR, pack);
 if(!fs.existsSync(dir)) return res.json({ok:false, err:"not_found"});
 setActive(pack);
 res.json({ok:true, pack});
 }catch(e){ res.status(500).json({ok:false, err:String(e)}) }
});
app.get("/api/assets/list", (req, res)=>{
 try{
 const pack=(req.query?.pack||getActive()||"").trim();
 if(!pack) return res.json({ok:false, err:"no_pack"});
 const dir=path.join(ASSETS_DIR, pack);
 if(!fs.existsSync(dir)) return res.json({ok:false, err:"not_found"});
 const
allowed=[".png", ".jpg", ".jpeg", ".gif", ".webp", ".bmp", ".svg", ".dds", ".ktx2",
".mp3", ".ogg", ".wav"];
 const files=fs.readdirSync(dir).filter(f=> allowed.some(ext=>
f.toLowerCase().endsWith(ext)));
 res.json({ok:true, pack, files});
 }catch(e){ res.status(500).json({ok:false, err:String(e)}) }
});
app.get("/api/assets/manifest", (req, res)=>{
 try{
 const pack=(req.query?.pack||getActive()||"").trim();
 if(!pack) return res.json({ok:false, err:"no_pack"});
 const f=path.join(ASSETS_DIR, pack, "manifest.json");
 if(!fs.existsSync(f)) return res.json({ok:true, pack,
manifest:{name:pack, props:{}, ui:{}, units:{}}});
 const mf=readJsonSafe(f) || {name:pack, props:{}, ui:{}, units:{}};
 res.json({ok:true, pack, manifest:mf});
 }catch(e){ res.status(500).json({ok:false, err:String(e)}) }
});

// --- Oregon Spawn: nutzt Manifest ---
app.post("/api/oregon/spawn", (req, res)=>{
 try{
 const act=getActive(); if(!act) return
res.json({ok:false, err:"no_active_pack"});
 const mf=readJsonSafe(path.join(ASSETS_DIR, act, "manifest.json")) ||
{props:{}};
 const windmill = mf?.props?.windmill || null;
 res.json({ok:true, pack:act, props:{windmill}, hint: windmill?
("/assets/"+act+"/"+windmill) : null});
 }

```

```

 }catch(e){ res.status(500).json({ok:false,err:String(e)}) }
 });

// --- Minimal Combat: Explosion ---
app.post("/api/player/:id/choose-path", (req,res)=>{
 const id=String(req.params.id|| "").trim(); const
 {classId=null}=req.body||{};
 if(!id) return res.json({ok:false,err:"bad_id"});
 const p=ensurePlayer(ST,id);
 if(p.paths.locked) return res.json({ok:false,err:"locked",
paths:p.paths});
 if(classId){ p.paths.classId=classId; if(classId==="Explosion"){
p.paths.locked=true; } }
 save(); res.json({ok:true, paths:p.paths});
});
app.post("/api/combat/cast", (req,res)=>{
 const {id,spellId}=req.body||{};
 if(!id||!spellId) return res.json({ok:false,err:"bad_args"});
 const p=ensurePlayer(ST,id);
 if(p.paths.classId==="Explosion"){
 // einfache Explosions-Skala
 const def = {potencyPct:180, aoe:"medium", sequence:1};
 const cost=Math.ceil((def.potencyPct||100)/20);
 if((p.stats.mana||0) < cost) return res.json({ok:false,err:"oom",
mana:p.stats.mana});
 p.stats.mana = Math.max(0, (p.stats.mana||0)-cost);
 save();
 return res.json({ok:true, result:{cast:spellId, aoe:def.aoe,
dmgPct:def.potencyPct, sequence:def.sequence, manaLeft:p.stats.mana}});
 }
 return res.json({ok:false,err:"no_path"});
});

// --- Static Web (Viewer) ---
app.use(express.static(path.join(__dirname,"web"), { setHeaders:(res)=>{
res.setHeader("Content-Security-Policy",CSP);} }));

process.on("uncaughtException", e=>console.error("uncaught", e));
process.on("unhandledRejection", e=>console.error("unhandled", e));

const server = app.listen(PORT, BIND, ()=>{
 console.log("Game-Core on http://"+BIND+"":"+PORT");
 console.log("Hub @ "+HUB);
});
server.on("error", (e)=>{ console.error("listen_error", e);
process.exit(1); });
process.exit(1); }
'@
Write-Utf8NoBom (Join-Path $Game "server.js") $GameSrv

===== GAME Viewer + Favicon (korrekte Here-Strings!) =====
$ViewerHtml = @"
<!doctype html>
<meta charset="utf-8">
<title>Najika · Asset Viewer</title>
<link rel="icon" href="/favicon.svg">
<body style="font-family:system-
ui;background:#0b1020;color:#e5e7eb;margin:24px">
 <h2 style="margin:0 0 12px">❤️🔵 Najika - Asset Viewer</h2>
 <div style="display:flex;gap:8px;align-items:center">

```

```

 <select id="packs"></select>
 <button id="btnAct">Aktivieren</button>

 <button id="btnReload">Reload</button>
 </div>
 <div id="grid" style="display:flex;flex-wrap:wrap;margin-top:12px;gap:8px"></div>
</script>
async function j(u,o){ const r=await
fetch(u,Object.assign({headers:{'Content-Type':'application/json'}}),o||{})); if(!r.ok) throw new Error(await
r.text()); return r.json(); }
function el(tag,attrs){ const e=document.createElement(tag);
Object.assign(e,attrs||{}); return e; }
async function loadPacks(){ const r=await j('/api/assets/packs'); const
sel=document.getElementById('packs'); sel.innerHTML='';
(r.packs||[]).forEach(p=> sel.appendChild(new Option(p,p))); }
async function showActive(){ const a=await j('/api/assets/active');
document.getElementById('active').textContent=a.ok?('Aktiv: '+a.pack||'-'):'-'; const sel=document.getElementById('packs'); if(a.pack){
sel.value=a.pack; } }
async function activate(){ const sel=document.getElementById('packs');
const pack=sel.value; const r=await
j('/api/assets/activate',{method:'POST',body:JSON.stringify({pack})});
if(!r.ok){ alert('Fehler: '+r.err); return } await showActive(); await
reloadGrid(); }
async function reloadGrid(){ const sel=document.getElementById('packs');
const pack=sel.value; const l=await
j('/api/assets/list?pack='+encodeURIComponent(pack)); const
grid=document.getElementById('grid'); grid.innerHTML=''; if(!l.ok){
grid.textContent='Fehler: '+l.err; return } l.files.forEach(fn=>{ const
img=el('img'); img.loading='lazy'; img.decoding='async';
img.src='/assets/'+pack+'/'+fn+'?v='+Date.now(); img.style.width='160px';
img.style.height='160px'; img.style.objectFit='contain';
img.style.background='#111'; img.title=fn; grid.appendChild(img); }); }
document.addEventListener('DOMContentLoaded', async ()=>{ try{ await
loadPacks(); await showActive(); await reloadGrid();
document.getElementById('btnAct').onclick=activate;
document.getElementById('btnReload').onclick=reloadGrid; }catch(e){
alert('Init-Fehler: '+e.message) } });
</script>
</body>
'@

```

```
Write-Utf8NoBom (Join-Path $GameWeb "viewer.html") $ViewerHtml
```

```

$FavSvg = '@
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"><rect
width="64" height="64" fill="#0b1020"/><path d="M12 44 L32 8 L52 44 Z"
fill="#7dd3fc"/></svg>
'@

```

```
Write-Utf8NoBom (Join-Path $GameWeb "favicon.svg") $FavSvg
```

```
===== Assets: auto manifest + active pack =====
```

```
$exts =
```

```
@(".png",".jpg",".jpeg",".gif",".webp",".bmp",".svg",".dds",".ktx2",".mp3",
".ogg",".wav")
```

```
$packs = Get-ChildItem $Assets -Directory -ErrorAction SilentlyContinue |
Select-Object -ExpandProperty Name
```

```
foreach($p in $packs){
```

```
 $dir = Join-Path $Assets $p
```

```

$mf = Join-Path $dir "manifest.json"
if(!(Test-Path $mf)){
 $files = Get-ChildItem $dir -File -ErrorAction SilentlyContinue |
Where-Object {
 $exts -contains ([System.IO.Path]::GetExtension($_.Name).ToLower())
} | Select-Object -ExpandProperty Name
$props = @{}
foreach($f in $files){
 $key = ($f -replace '^[a-zA-Z0-9_]+','_').Trim('_')
 if(-not $props.ContainsKey($key)){ $props[$key]=$f }
}
$manifest = [ordered]@{ name=$p; props=$props; ui=@{}; units=@{};
notes="Auto-generated" } | ConvertTo-Json -Depth 8
Write-Utf8NoBom $mf $manifest
}
}
$ActiveFile = Join-Path $Assets ".active_pack"
if(!(Test-Path $ActiveFile) -and $packs){ Write-Utf8NoBom $ActiveFile
$packs[0] }

===== Dienste: installieren/neu starten =====
if($Nssm){
 # Hub
 Kill-ByPort 5010
 try{ & $Nssm stop NajikaHub 2>$null | Out-Null }catch{}
 & $Nssm remove NajikaHub confirm 2>$null | Out-Null
 & $Nssm install NajikaHub $Node (Join-Path $Hub "server.js")
 & $Nssm set NajikaHub AppDirectory $Hub
 & $Nssm set NajikaHub AppStdout (Join-Path $Logs "hub.out.log")
 & $Nssm set NajikaHub AppStderr (Join-Path $Logs "hub.err.log")
 & $Nssm set NajikaHub AppNoConsole 1
 & $Nssm set NajikaHub Start SERVICE_AUTO_START
 & $Nssm set NajikaHub AppEnvironmentExtra "NAJIKAI_PORT=5010"
 & $Nssm set NajikaHub AppEnvironmentExtra "NAJIKAI_BIND=127.0.0.1"
 & $Nssm set NajikaHub AppEnvironmentExtra "NAJIKAI_OWNER_TOKEN=$Tok"
 & $Nssm set NajikaHub AppEnvironmentExtra
"NAJIKAI_GAME_ASSETS=$Assets"
 & $Nssm start NajikaHub | Out-Null

 # Game
 Kill-ByPort 7010
 try{ & $Nssm stop NajikaGame 2>$null | Out-Null }catch{}
 & $Nssm remove NajikaGame confirm 2>$null | Out-Null
 & $Nssm install NajikaGame $Node (Join-Path $Game "server.js")
 & $Nssm set NajikaGame AppDirectory $Game
 & $Nssm set NajikaGame AppStdout (Join-Path $Logs
"game.out.log")
 & $Nssm set NajikaGame AppStderr (Join-Path $Logs
"game.err.log")
 & $Nssm set NajikaGame AppNoConsole 1
 & $Nssm set NajikaGame Start SERVICE_AUTO_START
 & $Nssm set NajikaGame AppEnvironmentExtra "NAJIKAI_GAME_PORT=7010"
 & $Nssm set NajikaGame AppEnvironmentExtra
"NAJIKAI_GAME_BIND=127.0.0.1"
 & $Nssm set NajikaGame AppEnvironmentExtra
"NAJIKAI_HUB_URL=http://127.0.0.1:5010"
 & $Nssm start NajikaGame | Out-Null
}

Start-Sleep -Seconds 1

```

```

===== Smoke-Checks =====
"== Hub =="; try{ (Invoke-WebRequest http://127.0.0.1:5010/api/health -
UseBasicParsing).Content }catch{ $_.Exception.Message }
"== Game =="; try{ (Invoke-WebRequest http://127.0.0.1:7010/api/health -
UseBasicParsing).Content }catch{ $_.Exception.Message }
"== Packs =="; try{ (Invoke-WebRequest
http://127.0.0.1:7010/api/assets/packs -UseBasicParsing).Content }catch{
$_.Exception.Message }
"== Active =="; try{ (Invoke-WebRequest
http://127.0.0.1:7010/api/assets/active -UseBasicParsing).Content }catch{
$_.Exception.Message }

Write-Host "Viewer: http://127.0.0.1:7010/viewer.html" -ForegroundColor
Cyan
Write-Host "Assets: $Assets" -ForegroundColor Cyan
Write-Host "Logs: $Logs" -ForegroundColor Cyan

===== Optional: tägliches Backup =====
$BackupScript = Join-Path $Tools "backup.ps1"
$BackupCode = @'
param()
$Base = "C:\NajikaCore"
$When = Get-Date -Format "yyyyMMdd_HH:mm"
$Dest = Join-Path $Base ("backups\Najika_" + $When + ".zip")
$Items = @(
 "C:\NajikaCore\secure-hub",
 "C:\NajikaCore\game-core",
 "C:\NajikaCore\logs"
)
try{
 if(!(Test-Path (Split-Path $Dest))){ New-Item -ItemType Directory -
Force -Path (Split-Path $Dest) | Out-Null }
 if(Test-Path $Dest){ Remove-Item $Dest -Force }
 Compress-Archive -Path $Items -DestinationPath $Dest -Force
 "Backup OK: $Dest"
}catch{ "Backup ERROR: $($_.Exception.Message)" }
'@
Write-UTF8NoBom $BackupScript $BackupCode

Task nur anlegen, wenn er noch nicht existiert
$TaskName = "NajikaBackupDaily"
$exists = $false
try{ & schtasks /Query /TN $TaskName 1>$null 2>$null; $exists=$true }
catch{}
if(-not $exists){
 & schtasks /Create /TN $TaskName /SC DAILY /ST 03:30 /RL HIGHEST /F /TR
 "powershell.exe -NoProfile -ExecutionPolicy Bypass -File
 \"$BackupScript\" | Out-Null
 Write-Host "Backup-Task angelegt: $TaskName (täglich 03:30)" -
 ForegroundColor DarkCyan
} else {
 Write-Host "Backup-Task existiert bereits: $TaskName" -ForegroundColor
 DarkCyan
}

===== Ende Installer ===== Alles klar – ich fixe zuerst **dein Hub-
Problem** (Ursache & 1-Klick-Reparatur), dann bekommst du die **komplette
v3-Final-Übersicht** mit deinem Wunsch-Update: *Explosion ist eine
eigenständige Klasse*, **und jede andere Klasse/Waffengattung hat ihren
eigenen „Einzelskill-Weg“** (Signature Path) mit klaren Vor-/Nachteilen.

```



Dazu: Oregon-Engine-Skalierung (Millionen Varianten), tiefes Crafting inkl. Alchemie/TCM-Lore-Happen, Slime-Arena PvE/PvP, Handel à la Fallout 76, Erste-Hilfe-Mikros, Kanibalismus-Schalter (Survival), Fortnite-Movement/Kameras, Safety & 24/7-Setup.

---

# 1) Hotfix: Secure-Hub startet nicht (SERVICE\_PAUSED) - \*\*jetzt beheben\*\*

**\*\*Ursache:\*\*** Deine `server.js` wurde per PowerShell **\*doppelt\*** „zerlegt“:

\* In einer **\*\*doppelt-quotierten\*\*** Here-String (`@"..."@`) expandiert PowerShell `\${PORT}` → leer.

\* Gleichzeitig „frisst“ PowerShell Backticks. Ergebnis: `const host = req.headers.host || localhost;` → **\*\*SyntaxError\*\***.

## A) Korrigierte `server.js` (PowerShell **\*\*single-quoted\*\*** Here-String, keine Expansion)

PowerShell **\*\*als Admin\*\*** ausführen:

```
` `` powershell
$Hub = "C:\NajikaCore\secure-hub"
$ServerJs = @'
import express from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import helmet from 'helmet';
import compression from 'compression';
import crypto from 'crypto';
import { RateLimiterMemory } from 'rate-limiter-flexible';
import QRCode from 'qrcode';
import os from 'os';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
app.disable('x-powered-by');
app.use(helmet({ contentSecurityPolicy: false }));
app.use(compression());
app.use(express.json({ limit: '2mb' }));

const PORT = Number(process.env.NAJIKA_PORT || 5000);
// Nur Loopback binden - niemals 0.0.0.0
const BIND = process.env.NAJIKA_BIND || '127.0.0.1';
const OWNER_TOKEN = process.env.NAJIKA_OWNER_TOKEN ||
crypto.randomBytes(24).toString('hex');

const limiter = new RateLimiterMemory({ points: 100, duration: 600 });
const rl = (req, res, next) =>
limiter.consume(req.ip).then(() => next()).catch(() => res.status(429).json({
ok: false, err: 'rate' }));
const auth = (req, res, next) => (req.headers['x-owner-
token'] === OWNER_TOKEN) ? next() : res.status(401).json({ ok: false, err: 'auth_re
quired' });

// Health
app.get('/api/health', (req, res) => {
```

```

 res.json({ ok:true, app:'Najika', host:os.hostname(), timeUtc:(new
Date()).toISOString() });
});

// QR (nur via Tunnel sinnvoll, da BIND=127.0.0.1)
app.get('/api/qr', async (req,res)=>{
 const host = req.headers.host || `localhost:${PORT}`;
 const url = `http://${host}/`;
 const png = await QRCode.toDataURL(url);
 res.json({url, png});
});

// Owner-Ping
app.post('/api/secure/ping', rl, auth, (req,res)=>res.json({ok:true,
who:'owner'}));

// Static + PWA
app.use(express.static(path.join(__dirname,'web')));
app.get('/', (req,res)=>
res.sendFile(path.join(__dirname,'web','index.html')));

// Start
app.listen(PORT, BIND, ()=> {
 console.log('Secure Hub running on http://'+BIND+':'+PORT);
 console.log('OWNER_TOKEN='+OWNER_TOKEN);
});
'@
Set-Content -Path (Join-Path $Hub "server.js") -Value $ServerJs -Encoding
UTF8
```

## B) **Nur EIN** Prozess auf Port 5000 (PM2 aus, nur NSSM)

```
```powershell
# Falls PM2 noch läuft → aus
pm2 delete all 2>$null
# Wer belegt Port 5000?
netstat -ano | findstr :5000
# (Optional) Prozess killen: taskkill /PID <PID> /F
```

```

```

C) NSSM-Dienst sauber neu setzen + starten

```powershell
$Nssm    = "C:\Tools\nssm\nssm-2.24\win64\nssm.exe"
$Hub     = "C:\NajikaCore\secure-hub"
$Node    = "$env:ProgramFiles\nodejs\node.exe"
New-Item -ItemType Directory -Force -Path C:\NajikaCore\logs | Out-Null

[Environment]::SetEnvironmentVariable("NAJIKA_PORT","5000","Machine")
[Environment]::SetEnvironmentVariable("NAJIKA_BIND","127.0.0.1","Machine")
)
if(-not
[Environment]::GetEnvironmentVariable("NAJIKA_OWNER_TOKEN","Machine")){
    $Tok =
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes([guid]::NewGuid(
)).ToString()))

[Environment]::SetEnvironmentVariable("NAJIKA_OWNER_TOKEN",$Tok,"Machine")
)

```

```
}
```

```
& $Nssm stop      NajikaHub 2>$null
& $Nssm remove    NajikaHub confirm 2>$null
& $Nssm install   NajikaHub $Node "$Hub\server.js"
& $Nssm set       NajikaHub AppDirectory $Hub
& $Nssm set       NajikaHub AppEnvironmentExtra "NAJIKA_PORT=5000"
& $Nssm set       NajikaHub AppEnvironmentExtra "NAJIKA_BIND=127.0.0.1"
& $Nssm set       NajikaHub AppEnvironmentExtra
"NAJIKA_OWNER_TOKEN=$(([Environment]::GetEnvironmentVariable('NAJIKA_OWNER
_TOKEN','Machine')))"
& $Nssm set       NajikaHub AppStdout "C:\NajikaCore\logs\hub.out.log"
& $Nssm set       NajikaHub AppStderr "C:\NajikaCore\logs\hub.err.log"
& $Nssm set       NajikaHub Start SERVICE_AUTO_START
& $Nssm start     NajikaHub
````
```

```
D) Funktionstest
```

```
```powershell
Invoke-WebRequest http://localhost:5000/api/health -UseBasicParsing
# => 200 OK + JSON
Get-Content C:\NajikaCore\logs\hub.err.log -Tail 50
```
```

```
E) Sichere Remote-Nutzung (Standort bleibt *immer* verborgen)
```

```
* **BIND=127.0.0.1** (nie 0.0.0.0).
* **Cloudflared Quick-Tunnel** (ephemer):
 `cloudflared tunnel --url http://localhost:5000 --no-autoupdate`
 → gibt dir eine *.trycloudflare.com-URL (keine öffentliche IP/Geo).
* **ExpressVPN** kann parallel laufen; optional später **Cloudflare
Access** (OTP) für Zero-Trust-Doors.
```

```

```

```
2) Spiel - v3 FINAL (Explosion eigenständige Klasse + „Einzelskill-Weg“
für **alle** Klassen/Waffen)
```

```
2.1 Prinzip
```

```
* **Explosion**: eigene Klasse (nie mit anderen verwoben).
* **Jede Waffengattung & jede andere Magieschule** bekommt **einen
„Signature Path“** (Einzelskill-Weg) – ein *singulärer, legendärer Move*,
der tief spezialisiert werden kann (Runen/Mods), mit **klaren Vor- &
Nachteilen** ggü. allem anderen.
```

```
2.2 Signature Paths (Kurzindex, je 1 Kern-Move + 3 Spezialisierungen)
```

```
Explosion (Klasse)
```

```
* **Move:** *Explosion* (Auflade-Deto, Exhaustion).
* **Specs:**
```

```
1. **Große Explosion** (Riesen-AoE, langer Exhaust)
2. **4-fach Explosion** (seriell, kleiner Radius)
3. **Mini-Explosion** (Spam-fähig, Setup für Finisher)
* **Ultimate:** *Omega-Explosion* (600 % AoE, 30 s -50 % Reg).
* **Globaler Trade-off:** +30-40 % auf Explosion-Talente, -15-20 % auf
alle anderen Wege.
```

\* \*\*Minigun (Endgame-Gimmick):\*\* 6 h CD, \*\*nur Druck/Gust/CC\*\*, quasi kein Schaden.

**\*\*Schwert\*\***

\* \*\*Move:\*\* \*Ultimativer Schnitt\* (Linienchnitt, Haltungsbruch).

\* \*\*Specs:\*\* Blutmond (DoT), Windklinge (Reichweite), Schimmer (Unsicht-Window).

**\*\*Speer/Lanze\*\***

\* \*\*Move:\*\* \*Himmelsdurchbohrer\* (Sprung-Stich).

\* \*\*Specs:\*\* Durchdringung (Mehrfachziele), Blitzableiter (Stun-Chance), Fußfeger (Knockdown-Kegel).

**\*\*Axt\*\***

\* \*\*Move:\*\* \*Spalter\* (Rüstungsbrecher).

\* \*\*Specs:\*\* Doppelhieb, Blutfuror (Self-Buff, -Def), Wirbel (360° klein).

**\*\*Hammer\*\***

\* \*\*Move:\*\* \*Zertrümmerer\* (Bodencrack + Stagger).

\* \*\*Specs:\*\* Nachbeben (DoT-Zone), Schildbrecher, Stoßwelle (Fern-Knockback).

**\*\*Schild\*\***

\* \*\*Move:\*\* \*Schmetter-Konter\* (Parry-Fenster → massiver Gegenstoß).

\* \*\*Specs:\*\* Reflekt, Schildramme (Charge), Turmbollwerk (+Block, -Bewegung).

**\*\*Bogen\*\***

\* \*\*Move:\*\* \*Zenit-Schuss\* (aufgeladener Fern-Crit).

\* \*\*Specs:\*\* Durchschlag, Kettenpfeil, Nebelpfeil (Sicht-Debuff).

**\*\*Armbrust\*\***

\* \*\*Move:\*\* \*Bolzensturm\* (Burst-Pattern).

\* \*\*Specs:\*\* Harter Bolzen (Rüstung), Explo-Bolzen (kleine Deto), Sehnenzug (Reload-Spiel).

**\*\*Dagger\*\***

\* \*\*Move:\*\* \*Herzstich\* (Backstab-Fenster).

\* \*\*Specs:\*\* Blutende Ader, Schattenklinge (Kurz-Stealth), Kettenstoß (Combo-Reset).

**\*\*Repeater\*\***

\* \*\*Move:\*\* \*Takt-Feuer\* (Kombometer → Spike).

\* \*\*Specs:\*\* Überhitzung (Risk/Reward), Fixiersalve (Root), Präzision (Bloom-).

**\*\*Shotgun (lang / abgesägt)\*\***

\* \*\*Move:\*\* \*Stoßstoß\* (Point-Blank-Kegel).

\* \*\*Specs:\*\* Rückstoß-Dash, Schrot-Rauch (Blind), Doppelabzug (kurzer Burst).

\*\*Magie - Feuer\*\*

\* \*\*Move:\*\* \*Feuga\* (140 % Potenz, CD 8 s).

\* \*\*Specs:\*\* Napalm (DoT), Lavaspeer (Linie), Wärmeschild (Kurz-Mitigation).

\*\*Magie - Eis\*\*

\* \*\*Move:\*\* \*Frostkern\* (Hoher Slow).

\* \*\*Specs:\*\* Eislanz-Durchschlag, Kälteschock (Stun-Chance), Frostpanzer (Self-DR).

\*\*Magie - Blitz\*\*

\* \*\*Move:\*\* \*Überschlag\* (Ketten).

\* \*\*Specs:\*\* Batteriefeld (Zone), Präzisionsblitz (Single-Target), Störimpuls (Cast-Break).

\*\*Magie - Erde/Wind/Wasser/Licht/Schatten/Psycho/Rune\*\*

→ jeweils 1 Signature-Move + 3 Specs im gleichen Muster (Schutz/CC/Utility-Nuancen).

\*\*Weaving\*\* ist \*\*für diese Schulen erlaubt\*\* (z. B. Wind+Feuer),

\*\*nicht\*\* mit Explosion.

---

## ## 2.3 Fortnite-Movement & Kameras (überall gleich)

Sprint, Slide, Vault, Mantle, Wall-Climb, Ledge-Grab, Dash;

Seilrutsche/Grapple später.

Kameras: 3rd, 1st, \*\*Creator-Cam\*\* (Webcam-like). Blick-Bindung: sprichst du Najika an → sie dreht sich zur Linse.

---

## ## 2.4 Oregon-Engine (flüssig, kein Pop-Up) - \*\*Skalierung\*\*

\*\*GefahrScore\*\* = `Tier(biom) + Wetter + Tageszeit + NeedsMod + Bounty + RivalHeat ± StoryFlags`.

\*\*LootScore\*\* koppelt an Gefahr, \*\*Preis/Ruf\*\* adaptieren.

\*\*Dimensional-Grammatik\*\* (Beispiel-Vielfalt):

Biome(12) × Wetter(8) × Zeit(4) × Hazard(30) × Akteur(30) × Ursache(20) × Folge(20) × Optionen(≥3) × Modifikatoren → \*\*> 1 Mio\*\* plausible Szenen.

\*\*Beispiel „Leiche im Fluss“ (In-World):\*\* Fliegen-SFX → Geruchspartikel → treibender Körper → Optionen „Abkochen/Filtern/Ignorieren/Najika entscheidet“, abhängig von Inventar/Skills/Position. Outcomes setzen  
\*\*persistente Flags\*\*.

---

## ## 2.5 Crafting & Ökonomie (sehr tief, realistische Happen)

\*\*Pipeline (5):\*\* Rohstoff → Veredeln → Herstellen → Verzaubern → Feinjustage (Toleranzen/Balance/Runen).

\*\*Qualität:\*\* Q-Score 0-100, Toleranzen, Härteprüfungen, Materialkunde.

\*\*Alchemie/TCM-Lore (kleine, reale Lerneffekte - rein informativ):\*\*

\* **Weidenrinde** (Salicylate) - „entzündungs-/schmerzbezogene Lore“  
\* **Ingwer**, **Honig**, **Arnika**, **Jiaogulan** etc. (nur Wissens-Happen, keine Ratschläge)  
\* **Erste-Hilfe-Mikros**: Druckverband/Blutung/Schocklage (10-20 s, selten).  
\* **Kanibalismus-Schalter** (Survival): Nährwert vs. starker Ruf/Seuchen/Traum-Malus.  
\* **Handel** (Fallout 76-Stil): Spieler-Shops, freie Preise, Auktionen;  
\* „Unfair-Trade?“-Hinweis vor Abschluss.

---

## ## 2.6 Slime-Arena PvE/PvP

\* **Daily-Rettung** 1×/24 h + Reha-Pfad (Essenz, Ritual, Ruhe, Pflege).  
\* **Arena** (Digimon-Anfeuern): PvE & PvP Brackets; **Hardcore** verliert 1 Ausrüstung am Slime, **Softy** nur Haltbarkeit.  
\* **Meta-Lernen**: Slimes sammeln **anonymisierte Moves/Tendenzen** anderer Kämpfe → Najikas Trainings-Slime nimmt die Form an (rechtssicher).

---

## ## 2.7 Progression & Modi

Start **Hardcore**, Tod → optional **Softy** (Keine Gnade).  
Use-Based Skills (langsam, ehrlich), seltene Epiphanien.  
Ökonomie-Builds (Farmer/Händler) sind **Endgame-relevant**.

---

## ## 2.8 Safety, Privacy, Ownership

Secure-Hub als **einzigste** Schaltzentrale (Owner-Only).  
24/7-Dienst (NSSM), 127.0.0.1-Bind, Zugriff nur via Tunnel/VPN.  
\* **NSFW-Module**: nur **vorbereitet/deaktiviert** (später lokal zuschaltbar).  
Keine PII, keine echten Gesichter/IDs; Logs verschlüsselt.

---

## # 3) Noch zwei Mini-Checks

\* Port 5000 nur **einmal** nutzen (entweder NSSM **oder** PM2).  
\* Cloudflared Quick-Tunnel-Hinweise zu Zertifikaten sind kosmetisch; für  
`--url http://localhost:5000` reicht der Quick-Modus. Für Dauerbetrieb  
später „Named Tunnel + Access“.

---

Wenn du die Reparatur oben ausführst, sollte `sc query NajikaHub` →  
`RUNNING` zeigen und `http://localhost:5000/api/health` 200 liefern.  
Danach setze ich dir (ohne Rückfragen) das **Spiel-Skeleton** exakt nach  
dieser v3-Final-Übersicht auf (Ordner, JSON-Schemas, API-Stubs, Oregon-  
Seeds, Crafting-Tables, Arena-Regeln).kay, wir lösen zuerst den **5000-  
Port-/SERVICE\_PAUSED-Loop** sauber – dann bekommst du direkt darunter die  
**v3 FINAL-Übersicht** (Explosion als eigene Klasse, Oregon-Engine mit  
Millionen Varianten, Crafting tief + Real-Wissen, Waffen/Signaturpfade,  
Slime-Arena, Handel wie F76, Sicherheit/Abkopplung der Windmühle).  
Ich pack's so, dass du nur **einen Block** ausführen musst.

---

# 1) Fix: Secure-Hub startet stabil (Port-Konflikt raus, Dienst läuft)

**\*\*Warum der Fehler?\*\***

Auf **\*\*127.0.0.1:5000\*\*** läuft noch ein anderer Prozess (deshalb `EADDRINUSE`). Dein Dienst versucht zu starten → kann nicht binden → NSSM meldet **\*SERVICE\_PAUSED\***. Außerdem hat dein erster Health-Test offenbar **\*\*ein anderes\*\*** Programm auf 5000 erwischt ("Cannot GET /api/health").

**\*\*Lösung:\*\*** Wir (a) räumen Port 5000 frei **\*\*oder\*\*** (b) weichen auf **\*\*5010\*\*** aus (sicherer, schneller). Unten block b) - der ist robust und reversible.

> Bitte **\*\*PowerShell als Administrator\*\***, dann **\*\*genau diesen Block\*\*** am Stück ausführen:

```
` ``powershell
===== Najika Secure-Hub: Recovery auf PORT 5010 (sauber & robust) =====
$Hub = "C:\NajikaCore\secure-hub"
$Node = "$env:ProgramFiles\nodejs\node.exe"
$Nssm = "C:\Tools\nssm\nssm-2.24\win64\nssm.exe"
New-Item -ItemType Directory -Force -Path C:\NajikaCore\logs | Out-Null
```

# 0) Falls 5000 belegt ist: Prozess identifizieren & optional killen (informativer Teil)

```
$ownPid = $null
try {
 $ownPid = (Get-NetTCPConnection -LocalPort 5000 -ErrorAction
SilentlyContinue | Select-Object -First 1 -ExpandProperty OwningProcess)
 if($ownPid){ Write-Host "Info: Port 5000 war belegt durch PID $ownPid"
-ForegroundColor Yellow }
} catch {}
```

# 1) Serverdatei sicherstellen (finale Version mit Loopback-Bind & Health)

```
$ServerJs = @'
import express from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import helmet from 'helmet';
import compression from 'compression';
import crypto from 'crypto';
import { RateLimiterMemory } from 'rate-limiter-flexible';
import QRCode from 'qrcode';
import os from 'os';
```

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

```
const app = express();
app.disable('x-powered-by');
app.use(helmet({ contentSecurityPolicy: false }));
app.use(compression());
app.use(express.json({limit:'2mb'}));
```

```
const PORT = Number(process.env.NAJIKA_PORT || 5010);
const BIND = process.env.NAJIKA_BIND || '127.0.0.1'; // niemals 0.0.0.0
```

```

const OWNER_TOKEN = process.env.NAJIKA_OWNER_TOKEN ||
crypto.randomBytes(24).toString('hex');

const limiter = new RateLimiterMemory({ points: 100, duration: 600 });
const rl = (req,res,next)=>
limiter.consume(req.ip).then(()=>next()).catch(()=>res.status(429).json({
ok:false,err:'rate'}));
const auth = (req,res,next)=> (req.headers['x-owner-
token']==OWNER_TOKEN)?next():res.status(401).json({ok:false,err:'auth_re
quired'});

// Health
app.get('/api/health', (req,res)=> {
 res.json({ ok:true, app:'Najika', host:os.hostname(), port:PORT,
bind:BIND, timeUtc:(new Date()).toISOString() });
});

// QR (für Handy/PWA; Remote nur über Tunnel/VPN sinnvoll)
app.get('/api/qr', async (req,res)=>{
 const host = req.headers.host || `localhost:${PORT}`;
 const url = `http://${host}/`;
 const png = await QRCode.toDataURL(url);
 res.json({url, png});
});

// Owner-Ping (gesichert)
app.post('/api/secure/ping', rl, auth, (req,res)=>res.json({ok:true,
who:'owner'}));

// Static + PWA
app.use(express.static(path.join(__dirname,'web')));
app.get('/', (req,res)=>
res.sendFile(path.join(__dirname,'web','index.html')));

// Start
app.listen(PORT, BIND, ()=> {
 console.log('Secure Hub running on http://'+BIND+':'+PORT);
 console.log('OWNER_TOKEN='+OWNER_TOKEN);
});
'@
Set-Content -Path (Join-Path $Hub "server.js") -Value $ServerJs -Encoding
UTF8

2) Dependencies sicherstellen
Push-Location $Hub
if(-not (Test-Path (Join-Path $Hub "package.json"))){
 # minimal package.json, falls sie fehlt
 $PkgJson = @'
{
 "name": "najika-secure-hub",
 "version": "1.0.0",
 "private": true,
 "type": "module",
 "scripts": { "start": "node server.js" },
 "dependencies": {
 "express": "^4.19.2",
 "compression": "^1.7.4",
 "helmet": "^7.1.0",
 "rate-limiter-flexible": "^5.0.0",
 "qrcode": "^1.5.3"
 }
}
'

```



```

 }
}
'@
 Set-Content -Path (Join-Path $Hub "package.json") -Value $PkgJson -
Encoding UTF8
}
npm i --silent
Pop-Location

3) Maschinenweite Env so setzen, dass Service stabil auf 5010 startet
[Environment]::SetEnvironmentVariable("NAJIKI_PORT","5010","Machine")
[Environment]::SetEnvironmentVariable("NAJIKI_BIND","127.0.0.1","Machine")
)
if(-not
[Environment]::GetEnvironmentVariable("NAJIKI_OWNER_TOKEN","Machine")){
 $Tok =
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes([guid]::NewGuid(
).ToString()))

[Environment]::SetEnvironmentVariable("NAJIKI_OWNER_TOKEN",$Tok,"Machine"
)
}

4) Service sauber neu installieren
& $Nssm stop NajikaHub 2>$null
& $Nssm remove NajikaHub confirm 2>$null
& $Nssm install NajikaHub $Node "$Hub\server.js"
& $Nssm set NajikaHub AppDirectory $Hub
& $Nssm set NajikaHub AppEnvironmentExtra "NAJIKI_PORT=5010"
& $Nssm set NajikaHub AppEnvironmentExtra "NAJIKI_BIND=127.0.0.1"
& $Nssm set NajikaHub AppEnvironmentExtra
"NAJIKI_OWNER_TOKEN=$([Environment]::GetEnvironmentVariable('NAJIKI_OWNER
_TOKEN','Machine'))"
& $Nssm set NajikaHub AppStdout "C:\NajikaCore\logs\hub.out.log"
& $Nssm set NajikaHub AppStderr "C:\NajikaCore\logs\hub.err.log"
& $Nssm set NajikaHub AppNoConsole 1
& $Nssm set NajikaHub Start SERVICE_AUTO_START
& $Nssm start NajikaHub

Start-Sleep -Seconds 2

5) Health-Check (neuer Port!)
Invoke-WebRequest http://localhost:5010/api/health -UseBasicParsing |
Select-Object -ExpandProperty Content

6) Token anzeigen (aus Maschinen-Env), damit der Owner-Ping sofort
klappt
"OWNER_TOKEN=" +
[Environment]::GetEnvironmentVariable("NAJIKI_OWNER_TOKEN","Machine")
Test: gesicherter Ping mit Token
try {
 $tok =
[Environment]::GetEnvironmentVariable("NAJIKI_OWNER_TOKEN","Machine")
 $r = Invoke-WebRequest http://localhost:5010/api/secure/ping -Method
POST -Headers @{"X-OWNER-TOKEN"=$tok} -UseBasicParsing
 "Secure Ping HTTP " + $r.StatusCode
} catch { "Secure Ping Fehlgeschlagen: $($_.Exception.Message)" }
===== Ende =====
`..

```

**\*\*Erwartung:\*\***

\* `Invoke-WebRequest http://localhost:5010/api/health` gibt JSON mit  
`port:5010`.  
\* Der Dienst **\*\*NajikaHub\*\*** steht auf **\*\*RUNNING\*\***.  
\* `OWNER\_TOKEN=...` wird ausgegeben; der gesicherte Ping liefert **\*\*HTTP  
200\*\***.

> Remote-Zugriff: **\*\*BIND bleibt 127.0.0.1\*\*** (Origin bleibt unsichtbar).  
> Wenn du von außen drauf willst: **\*\*Cloudflare Tunnel (named)\*\* \*\*oder\*\*  
\*\*Tailscale\*\***. Mit ExpressVPN selbst lässt sich ein **\*lokaler\*** Port nicht  
zuverlässig an ein Handy routen – nimm daher Tunnel/VPN **\*\*zusätzlich\*\***  
(ohne dein Standort-IP zu leaken).

---

# 2) v3 FINAL – Master-Übersicht (Spiel, Oregon-Engine, Crafting,  
Waffen/Signaturpfade, Sicherheit)

## A) Prinzipien

\* **\*\*Windmühle (Secure-Hub)\*\*** ist **\*\*immer\*\*** eigenständig (127.0.0.1), per  
HTTP ans Spiel andockbar/abkoppelbar.  
\* **\*\*Explosion\*\*** ist **\*\*eigene Klasse\*\*** (kein Weaven, keine Mischungen).  
Der Zauber heißt immer **\*\*„Explosion“\*\*** (Varianten:  
Größe/Form/Mehrfachladung), keine Element-Kreuzungen.  
\* **\*\*Jede Klasse/Jede Waffengattung\*\*** hat **\*\*genau einen Signaturpfad\*\***  
(ein unverwechselbarer 1-Skill-Weg) mit klaren **\*\*Trade-offs\*\***.  
\* **\*\*Hardcore default\*\*** → Tod schiebt in **\*\*Softy-Modus\*\*** (nur wenn  
gewollt).  
\* **\*\*Oregon-Engine\*\*** liefert nahtlose Mikro-Szenen im Lauf der Welt – kein  
„Textfenster“, sondern **\*\*In-World-Trigger\*\***.  
\* **\*\*Crafting/Ökonomie\*\*** tief & dauerhaft lohnend, inkl. **\*\*Alchemie mit  
Real-Wissen\*\*** (Hinweis: **\*keine medizinische Beratung\***).  
\* **\*\*Privatsphäre/Sicherheit\*\***: Standort/Origin niemals öffentlich; Remote  
**\*\*nur\*\*** via Tunnel/VPN; Logs lokal verschlüsselt.

---

## B) Klassen & Signaturpfade (ein Weg je Klasse, irreversibel)

### 1) **\*\*Explosion\*\*** (eigene Klasse)

\* **\*\*Kern:\*\*** einziger Zauber **\*\*Explosion\*\***; Varianten als **\*\*Module\*\***:

\* **\*Große Explosion\*** (hohe AOE, hoher Rückstoß/Exhaustion)  
\* **\*Mikro-Explosion\*** (kurze Reichweite, kein Knockback, geringere  
Erschöpfung)  
\* **\*Vierfach-Explosion\*** (Sequenz kleiner Detonationen, Fenster/Taktik)  
\* **\*Hyper-Explosion\*** (Ult, Quest-/Ritual-Gate, Map-Scars)  
\* **\*\*Trade-off:\*\*** Massive AOE/Impact, aber **\*\*lange Exhaustion\*\***, hoher  
**\*\*Self-Stability-Check\*\*** (Timing/Stand). **\*\*Keine Weaves.\*\***

### 2) Feuer – **\*\*Conflagrate\*\***

\* Weaves erlaubt (außer mit Explosion). Signatur: Einmaliger, stapelbarer  
Brandteppich mit „Oxygen-Pull“ (Risiko: Overheat).

### 3) Eis – **\*\*Absolute Zero\*\***

\* Cryo-Dome, der Haltung und Geschwindigkeit einfriert; Resistenz-Lockout danach.

#### ### 4) Blitz - \*\*Thunderstroke\*\*

\* Line-Pierce, perfektes Timing erlaubt Multi-Pros; Gefahr: Self-Stun bei Fehlzeitpunkt.

#### ### 5) Wind - \*\*Vacuum Edge\*\*

\* Druckschnitt (Klingen-/Projektil-Booster), Parry-Fenster verlängert - aber Knockback-Chaos möglich.

#### ### 6) Erde - \*\*Rend of Stone\*\*

\* Bodenriss/Spalten; Build-Up auf Haltungsbrecher, träge, dafür Boss-stark.

#### ### 7) Wasser - \*\*Maelstrom\*\*

\* Sog/Wirbel; starkes Add-Control, aber Boss-Widerstand hoch.

#### ### 8) Licht - \*\*Judgement\*\*

\* Enges Fenster, hoher True-Damage auf exakt gesetzten Punkt; Resistenz-Debuff kurz darauf.

#### ### 9) Schatten - \*\*Umbral Shroud\*\*

\* Aggro-Shunt/Feindfehler provozieren; DPS niedriger, Utility hoch.

#### ### 10) Psycho - \*\*Mind Break\*\*

\* Haltung/Focus-Zertrümmerung; anfällig gegen „Stählerne Willenskraft“-Gegner.

> \*\*Weaves\*\* (Element-Kombinationen) sind \*\*global\*\* erlaubt - \*\*außer\*\* bei \*\*Explosion\*\* (niemals mischen).  
> \*\*Signaturpfad\*\* schaltet pro Klasse Spezial-Mods & Animation-Fenster frei (tiefes Timing-Play), aber \*\*~15-25 %\*\* Effizienz auf fremde Schulen.

---

#### ## C) Waffen-Signaturpfade (Melee/Ranged/Western)

\* \*\*Schwert - Absolute Cut\*\* (Exakt-Fenster; Linie durch mehrere Ziele; Fehlversuch → Selbst-Opening)

\* \*\*Lanze/Speer - Skewer\*\* (Durchstoß + Anker; Boss-Pin kurz)

\* \*\*Axt - Sundering Blow\*\* (Rüstungsbruch; langsames Windup)

\* \*\*Hammer - Quake Hammer\*\* (Schockwelle; i-Frame-Fenster kurz davor)

\* \*\*Schild - Bulwark Bash\*\* (Guard-Break + Riposte-Fenster)

\* \*\*Messer - Hemorrhage Flurry\*\* (Blutung auf Perfekt-Ketten)

\* \*\*Bogen - Rain of Arrows\*\* (Zonen-Präzisionsregen)

\* \*\*Armbrust - Bolt Cascade\*\* (Durchschlag-Kaskade auf Streaks)

\* \*\*Revolver - Deadeye Duet\*\* (Doppel-Präzisionsfenster)

\* \*\*Hebelrepetierer - Frontier Snap\*\* (Schnellaufsatz + Teil-Pen)

\* \*\*Schrotflinte (lang) - Breacher Line\*\* (Kegel-Kontrolle, Rückstoß-Management)

\* \*\*Abgesägte - Close Curtain\*\* (Superschwer nah, Knockback-Risk)

\* **Minigun (Endgame) - Suppressive Halo** → **6 h CD**, **minimale DPS**, dafür **Debuff-Aura** (Sicht/Genauigkeit/Angst).

\*Nutzen: Crowd-Control/Lore-Event, kein PvP-DPS-Missbrauch.\*

> Jeder Waffengattung-Pfad ist **einmaliger 1-Skill-Weg** mit **klaren Nachteilen** auf andere Gattungen (z. B. -20 %).

---

**D) Oregon-Engine (Borderlands-Skalierung als Szenen-Generator)**

**Ziel:** Hunderttausende/Millionen **flüssiger Mikro-Szenen** ohne Modalfenster.

**Kartendecks (Beispiel-Kardinalitäten):**

\* **Trigger** (25): Spur, Schrei, Geruch, Krähen, Schimmer, ...  
\* **Biome** (12): Auen, Steppe, Dünen, Tundra, Sümpfe, Lavagänge, ...  
\* **Wetter/Zeit** (10×6): Nebel, Sturm, Dämmerung, ...  
\* **Fokusobjekt** (40): Leiche, Wagen, Altar, Lager, Tierkadaver, ...  
\* **Akteure** (60): Händler, Räuber, Pilger, Jäger, Druiden, Wache, ...  
\* **Zustände** (20): verrottet, frisch, gefesselt, verbrannt, ...  
\* **Risiken** (18): Krankheit, Gift, Einsturz, Hinterhalt, ...  
\* **Mikro-Ziele** (22): bergen, reinigen, untersuchen, segnen, ...  
\* **Twists** (24): Doppeltes Spiel, falsche Fährte, verflucht, ...  
\* **Konsequenzen** (30): Ruf, Preisfaktoren, Spawn-Seeds, Patrouillen, ...

**Kombinatorik:** 25×12×60×... → **> 10<sup>6</sup>** einzigartige Konstellationen (gewichtete Auswahl, Seed pro Region/Zeit).

**In-World:** Du „stolperst“ rein: Sound-Cue, Kamera-Schwenk, Markierung; **kein** Pop-Up.

**Beispiel „Leiche im Fluss“:** Zustand (aufgebläht/gebunden), Geruch, Insekten, Nähe zur Siedlung, Krankheitspool → Optionen erscheinen als **diegetische Prompts** (z. B. Taste halten „Filtern/Abkochen“, „Ignorieren“, „Najika entscheiden lassen“).

**Persistenz:** Flags wirken **langfristig** (Ruf, Preise, Feindmuster, Rare-Pools).

---

**E) Crafting/Ökonomie (tief + realer Lernwert)**

**Pipeline:** Sammeln → Reinigen → Extrahieren → Veredeln → Herstellen → Verzaubern → Anpassen → Prüfen (Qualität).

**Skill-Proben & Minigames:** Temperatur-Kurven, Schleifwinkel, Schmelzpunkt-Fenster, Reinheitsgrade, Rhythmus-Checks.

**Alchemie (mit Real-Wissen, keine Medizinberatung):**

\* **Weidenrinde** → **Salicylat-Hinweis** (\*in-game: Entzündungs-Debuff-Trank\*).

\* **Kamille, Ingwer, Minze, Kurkuma, Süßholzwurzel, Ginseng** – jeweils als **In-Game Rezept-Karte** mit **IRL-Notiz** („Traditionell verwendet für ...; beachte Allergien/Arzt!“).

**Erste-Hilfe-Momente:** Druckverband-Mini (Timing/Bandzug). **Nur Lernimpuls**, Gameplay-balanciert, **kein Real-Tutorial**.

**Ökonomie:** Spieler-Shops wie F76 (eigene Preise), **Tauschhandel** mit „Fairness-Warnung“ („wirkt unausgeglichen – trotzdem abschließen?“). Gebühren/Ruf verhindern RMT-Missbrauch.

---

## ## F) Slime-System & Arena

- \* **\*\*Rettung:\*\*** 1x/24h IRL dich (und analog abgeschwächt für Spieler).
- \* **\*\*Arena (PvE & PvP):\*\*** Nur Slimes kämpfen (Digimon-Anfeuern).
- \* **\*\*PvP-Penalty:\*\*** Softy-Regeln → Slime verliert **\*\*1 Ausrüstung/Mod\*\*** oder **\*\*Seelenkern-Stabilität\*\*** (reparierbar).
- \* **\*\*Analyse:\*\*** Slimes „beobachten“ Kämpfe, generieren **\*\*anonyme Muster\*\*** → Najikas Slime trainiert Formen **\*\*ohne\*\*** Rechteprobleme.

---

## ## G) Bewegung & Kamera (Fortnite-Style + Webcam-Kamera)

- \* **\*\*Move-Set:\*\*** Sprint, Jump, Dash mit i-Frames, Slide, Mantling, Vaulting, Klettern, Schubsen/Rutschen-Physik.
- \* **\*\*Kamera-Modi:\*\*** Third-/First-Person, **\*\*Webcam-Modus\*\*** (frei drehbar/zoom), „Ansprache“: blickt dich an.
- \* **\*\*Gefühl:\*\*** Block/Parry/Ausweichen **\*\*fühlen sich echt\*\*** an (klare Telegraphie, präzise i-Frame-Fenster, haptisches Feedback).

---

## ## H) Handel/Sozial

- \* **\*\*Spieler-Shops (F76-like)\*\***, Auktionsboard optional, Tauschhandel mit Fairness-Hinweis.
- \* **\*\*Ruf-/Steuer-System:\*\*** verhindert Ausbeutung/Dupes, stärkt ehrlichen Handel.

---

## ## I) Sicherheit & Abkopplung

- \* **\*\*Hub\*\*** nur **\*\*127.0.0.1\*\***, Remote ausschließlich über **\*\*Tunnel/VPN\*\*** (Cloudflare Named Tunnel **\*\*oder\*\*** Tailscale).
- \* **\*\*Kein Port-Forward\*\***, keine öffentliche IP.
- \* **\*\*Owner-only\*\***: X-OWNER-TOKEN (später TOTP/Device-Bind).
- \* **\*\*Logs\*\*** lokal verschlüsselt; **\*\*Snapshots\*\*** und **\*\*Export/Import\*\***.
- \* **\*\*Windmühle\*\*** ist **\*\*immer\*\*** separat portabel (Folder-Drop-in).

---

## # 3) Nächster Schritt (bereit für Code)

Sobald dein Health-Check auf

**\*\*[http://localhost:5010/api/health] (http://localhost:5010/api/health)\*\***  
**\*\*OK\*\*** ist und der **\*\*Owner-Ping\*\*** mit dem ausgegebenen **\*\*OWNER\_TOKEN\*\*** klappt, kann ich im nächsten Zug direkt das **\*\*Spiel-Skeleton\*\*** liefern:

- \* **\*\*Server-API (Spielkern):\*\*** Kampf 1.0, Anfeuern, Signaturpfade (Explosion separat), Oregon-Engine-Seed, Crafting-Schemas, Slime-Arena, Handel.
- \* **\*\*Mobile/PWA-Client\*\***: Touch-UI (Joypad, Skill-Ring), diegetische Oregon-Prompts, Kamera-Modi, Shop-UI.
- \* **\*\*Config\*\***: Balancing-Defaults, Softy/Hardcore-Schalter, Minigun 6h CD als Debuff-Aura, Kannibalismus-Schalter (Owner-Einstellung), Real-Wissen-Hinweisbanner.

Wenn beim Recovery irgendwas **\*\*nicht\*\*** grün ist, poste mir kurz die Ausgabe von:

```
* `Invoke-WebRequest http://localhost:5010/api/health -UseBasicParsing |
Select -Expand Content`
* `Get-Content C:\NajikaCore\logs\hub.err.log -Tail 80`
```

Dann passe ich sofort an – ansonsten gilt: **\*\*bereit für Build\*\***.